

Architectural Roadmap Agentic Worldbuilding

Bridging Large Language Models and
Deterministic Spatial Manipulation in the ACME
Ecosystem



PREPARED FOR THE ACME OPEN SOURCE INITIATIVE |
SPRING 2026



Executive Summary

The transition from a GUI-driven point-and-click application to a fully autonomous, agentic worldbuilding system requires a massive re-engineering of how data moves, is validated, and is subsequently digested by artificial intelligence. We are bridging high-level reasoning modules—specifically Vision-Language Models (VLMs) and advanced Large Language Models (LLMs)—with the underlying `WorldBuilder.Shared` service layer. This document serves as the highly detailed, canonical roadmap for this architecture.

This project is not built in a vacuum. It heavily relies on the titans of the Asheron's Call open source reverse-engineering community. For our agent pipeline to function, we have deeply integrated **ACEmulator (ACE)** as our backend sanity check, **DatReaderWriter** to interact with the raw client data files at lightspeed, **ACViewer** to act as the visual bridge for our Vision Models, and the heavily datamined **LSD-Partial**

(Lifestoned) repository to provide the contextual "ground truth" to the AI.

"We are moving beyond procedural noise algorithms. We are building a system where an AI can visually 'see' the world, understand lore constraints, gracefully interact with a headless CLI pipe, and intelligently place a functioning town—down to the last merchant NPC and valid portal link."

What follows is an incredibly dense, 6-phase exploration into the technical systems, data structures, formatting schemas, and theoretical frameworks driving this agentic pipeline. We cover everything from the parsing of Quaternion matrices preventing agent gimbal-lock, to the massive ecosystem dependencies that feed the LLM's brain.

Phase 1: Service Layer Consolidation

CORE FOUNDATION

The core philosophy of the ACME terminal architecture is simple: GUI interactions and CLI interactions must use the strictly identical dependency-injected services. Decoupling view models from business logic was paramount.

When an agent issues a command to alter the world, it cannot rely on async UI threads or rendering frames to process its requests. Therefore, every single editing capacity of WorldBuilder was completely stripped of its graphical roots and compiled into pure mathematical headless modules under the `WorldBuilder.Shared` namespace.

The Dungeon Graph Extraction

The internal room cell linking logic and `PortalSnapper` natively existed in UI bound spaces. In the original paradigm, a user would drag a dungeon room, and the UI would calculate nearest-neighbor intersections based on mouse positioning. We extracted this into the isolated `IDungeonService`.

This service is tasked with calculating the mathematical alignment of portal nodes—verifying reciprocal exits, tracking quaternion translations, and aligning geometry—without ever invoking an Avalonia view model. When the LLM Agent decides it wants an underground catacomb, it passes an array of cell IDs. The `IDungeonService` takes over, attempting to weave the graph matrix mathematically, ensuring that if room A connects to Room B, Room B perfectly rotates its quaternion data so that the entrance polygon aligns with Room A's exit polygon.

Command Pattern Standardization

To ensure agents can fluidly stream batches of actions without synchronous API lockup, every interaction utilizes standard `CommandEngine.cs` implementations. An agent passes JSON via `stdin`, and it is routed directly to the engine.

```
// Sample stdin queue passed from the LLM
execution planner
// Notice how the agent must explicitly s
tate Quaternions (qw, qx, qy, qz)
[
  {"command": "add-object", "modelId": "0x0
2005A31", "lbX": 7, "lbY": 8, "x": 1050.5, "y": -200.2,
"z":
    45.0, "qw": 1.0, "qx": 0.0, "qy": 0.0, "q
z": 0.0},
  {"command": "raise", "x": 1050.0, "y": -2
00.0, "radius": 24.0, "delta": 5},
  {"command": "paint", "x": 1050.0, "y": -2
00.0, "radius": 24.0, "type": 5},
  {"command": "validate-all", "lbX": 7, "lb
Y": 8}
]
```

This queue system allows an agent to design an entire settlement in its own memory buffer, then dump the execution pipeline into the terminal in a single millisecond. The engine processes it deterministically, returning a line-by-line JSON payload indicating success, or specific error codes if a physical law was violated.

Phase 2: State Telemetry & Observation

AGENT VISION

An artificial agent cannot build effectively without acute spatial awareness. It doesn't have a monitor; it has text. Therefore, the terminal exposes high-fidelity, machine-readable telemetry of the DAT environment directly to the standard output buffer in JSON.

Quaternion Math for the Agent

Gimbal lock is a well-known vulnerability in spatial manipulation. If you tell an LLM to "rotate this building 90 degrees on the Y axis", standard Euler configurations run the extreme risk of overlapping axes,

destroying the agent's internal tracking of where the structure is actually pointing. To protect agent calculations, `WorldBuilder.Terminal` serializes all rotation vectors in 4D Quaternions instead of raw Euler angles.

When the LLM Agent attempts a precise 90-degree snap to align an armory with a castle wall, it manipulates q_w , q_x , q_y , q_z coordinates directly. The agent's backend context prompt includes the exact rotational math formulas necessary to calculate these four vectors, ensuring perfectly flawless wall alignments.

Heightmap Matrix Extraction

Raw terrain data of specific 24×24 unit cell grids are exported as 2D numeric matrices. By firing ``get-heightmap``, the agent receives the precise topography needed to evaluate slopes before laying road networks or dropping foundations.

```
// Response from get-heightmap
{
  "success": true,
  "command": "get-heightmap",
  "landblock": "0x0708",
  "gridSize": 9,
  "cellSize": 24,
  "heightsWorld": [
    [120.5, 122.3, 125.0, 128.1, 129.4, 131.
0, 131.5, 132.0, 132.5], // Row 0
    [119.5, 121.2, 123.8, 126.5, 128.9, 130.
1, 131.0, 131.8, 132.4], // Row 1
    // ... continues for 9 rows
  ]
}
```

This allows an agent to calculate variance. If the variance between Row 0, Index 0 and Row 1, Index 0 is too high, the agent logically concludes there is a steep cliff. It will inherently know to execute a smooth or lower command to flatten the terrain before attempting to drop a housing structure, mimicking human intuition.

Spatial Querying (Spherical Radius Checking)

Through commands like `query-radius`, agents use exact Euclidean distance formulas to probe the density of static objects. This guarantees an agent won't blindly drop an NPC spawn directly inside a dense thicket of trees or inside an existing structure. It queries the radius, parses the JSON return of all objects occupying that vector, and checks for overlaps.

Ecosystem Integration:

DatReaderWriter

BACKEND DEPENDENCY

None of this highly advanced command pipeline would be fast enough to support an LLM without the integration of **DatReaderWriter.Extensions**. This library is the foundational bed of our entire engine.

The standard methodology of interacting with Asheron's Call .dat files involves incredibly archaic binary parsing. `DatReaderWriter` abstractions allow our terminal to instantaneously pull `DBObj` classes. Specifically, the extensions library—with its `DatEasyWriter` interfaces—permits incredibly rapid prototyping of our modifications.

When the agent issues thousands of `raise`, `paint`, and `add-object` commands, the pipeline caches these into WorldBuilder's SQLite document tracking. Upon the `export` command, our systems bind directly into the `DataReaderWriter`, cloning headers, defragmenting spaces, and iterating via `writer.Save(landBlock)` to write the massive LLM generated zones back to the binary files in mere seconds. Without this performant library acting as the backend to our shared services, training an agent across hundreds of iterations would bottleneck at the hard-disk write phase.

Phase 3: Deterministic Validation Engine

CONSTRAINT SAFETY

LLMs are brilliant planners but notoriously poor mathematicians; left to their own devices, they will hallucinate coordinates. The headless `ValidationEngine.cs` is an unyielding gatekeeper designed to reject impossible geometries before they corrupt the in-memory DAT representation.



The Diagnostic Codes

We wrote 30 distinct validation rules separated into specific domains. The engine will literally map physical locations and spit out precise error codes to teach the agent exactly what it did wrong.

Code	Diagnostic Trigger	Validation Explanation & Engine Logic
LBK001/002	Out of Bounds	Object coordinates physically exceed the 192x192 landblock boundary constraints. The grid calculations instantly kick the object back to the agent for recalculation.
LBK003/004	Z-Axis Topographical Clamping	Checks if an agent placed an object deeply underwater below the terrain mesh, or floating in the sky (+500 units above heightmap). It pulls the current heightmap vector, subtracts the object's Z value, and throws the error if the absolute delta is unreasonable.

LBK005Zero-Scale
Object

Agent attempted to place an invisible, near-zero scale layout. This prevents "junk data" piling up inside the DAT due to floating point rounding errors in the neural network's outputs.

LBK006Degenerate
Quaternion

Quaternion length significantly diverges from 1.0. The agent messed up its normalization math, and the engine refuses to break the rendering pipeline.

TRN002Extreme Cliff
Detected

A raise command caused a shear vertical cliff > 100 units between adjacent 24x24 unit vertices, generating an un-walkable physical anomaly. This frequently occurs when an agent attempts to create flat plateaus and fails to slope the surrounding cells.

DNG003/004Broken Portal
Link

Mathematical sanity checks failed. The transform matrices of connection portal nodes are not perfectly inverse; or, an agent linked Cell A to Cell B, but Cell B lacks a reciprocal exit back to Cell A.

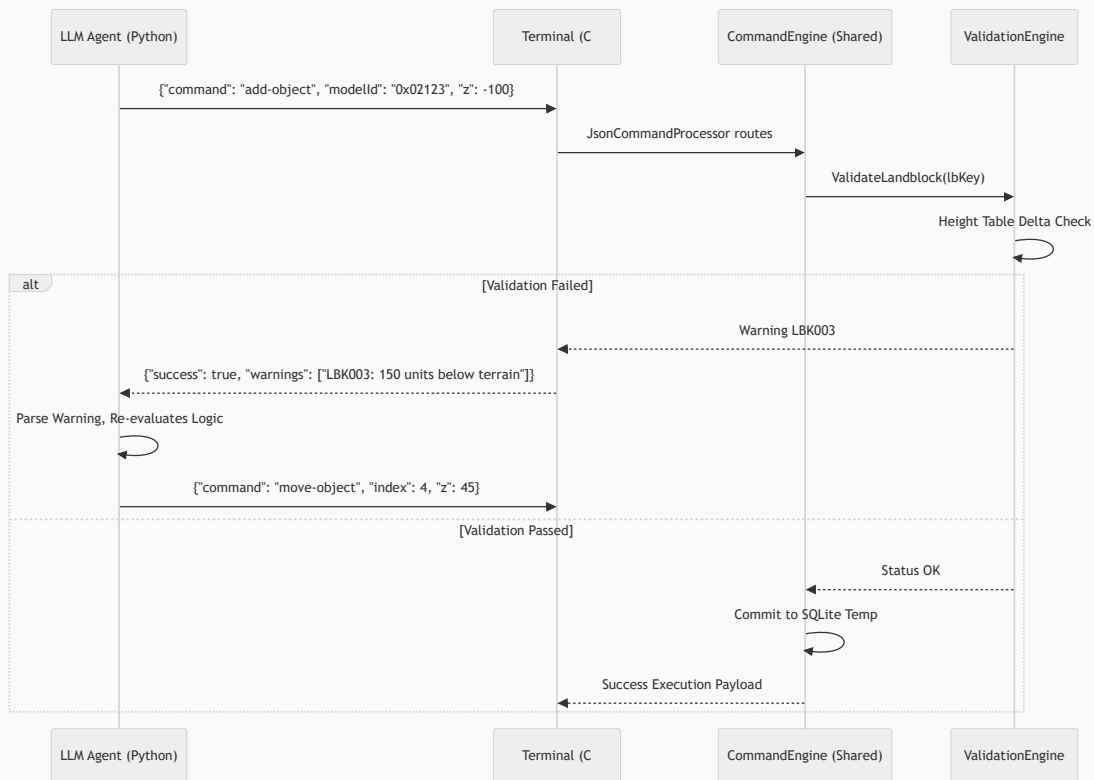
DNG011

Orphaned
Graph Node

Validates dungeon structure using Breadth-First Search (BFS). It starts at the drop point and spreads through every active portal. If there are cells physically present in the landblock that the BFS never touched, it flags them as unreachable orphans.

The system forces the Agent into a loop of responsibility. If it breaks the geometry, the JSON return yells at it. Standard LLM behavior will then parse the `warnings: ["LBK003"]` context, realize its Z-axis was hallucinated, and issue an immediate correction command to pull the object back to the ground.

The Agentic Feedback Loop



Architectural Diagram of the Validation Loop rejecting hallucinated parameters natively.

Ecosystem Integration:

ACEmulator (ACE)

GAME LOGIC SANDBOX

Validating geometry internally via the `ValidationEngine.cs` is one thing, but how do we validate narrative implementation and gameplay flow? That is where the **ACEmulator (ACE)** master repository integration occurs.

After an agent spins up a brand new custom dungeon—complete with dozens of interconnected rooms, correctly aligned quaternion models, and beautifully smoothed terrain entrances—we have to ensure it functions. The agent triggers an `export` command, packing the data.

We then utilize automated dockers bound to the ACEmulator core server. The generated DAT files are hot-swapped into a standalone, local ACE instance. The ACE platform acts as the ultimate integration sandbox. Our testing frameworks boot up headless client accounts, log into the ACEmulator instance via specific test hooks, and physically walk through the agent's generated world.

ACE handles lighting, monster pathing grids, dynamic spawning, and interaction rules. If the ACEmulator console throws NavMesh exceptions, or if our test bot gets permanently stuck inside a poorly rotated building mesh, the external testing pipeline flags the failure, wiping the landblock out of the agent's memory and demanding a total redesign.

Phase 4: Data Extraction & The Training Corpus

DEEP LEARNING

To train an agent to build like an original franchise developer, it requires a dataset of vanilla design patterns. Procedural placement via standard algorithms (like Wave Function Collapse) is fundamentally insufficient for rich narrative generation. We intend to fine-tune generative models over hundreds of epochs utilizing the literal geometry of Dereth.

Pattern Tokenization Engine

Our extraction scripts iterate through millions of bytes of `client_cell_1.dat` and `client_portal.dat` data. The algorithms catalog massive adjacency frequencies. *Example: How often is Model A (A Sho Lantern) placed within 5 Euclidean units of Model B (A Sho Marketplace)?* This statistical tokenizer writes directly to training datasets, converting a 3D world into a massive tokenized text document that an LLM can consume.

Ecosystem Integration: LSD-Partial (The Semantic Ground Truth)

Geometry alone doesn't convey meaning. A building is just a collection of intersecting bounding boxes unless an AI knows what dwells inside it. We have deeply integrated the **LSD-Partial** data repository (Lifestoned) to solve this.

The LSD-Partial dataset provides the ultimate semantic "contextual payload" for our agents. While WorldBuilder terminal tracks coordinate data, the `weenies` folder within LSD tracks the metaphysical data: creature stats, faction alignments, level tiers, merchant loot tables, and spell casting logic.

The `spawnMaps` folder maps these weenies directly to coordinate vectors. By feeding the LLM the overlap between WorldBuilder's terrain extraction logic and LSD's spawn tier mapping, the AI learns *contextually*. It learns that Banderling camps belong natively in mid-tier elevated regions. It understands that a level 50 magical merchant shouldn't be spawned outside a newbie starter town.

Asset Discovery & The "Screenshot Oracle"

Large Language Models are completely blind without visual grounding. Identifying an object by its hex code (`0x02001140`) provides no artistic value. They need to see the world.

This is where our integration with **ACViewer** comes in. We leverage ACViewer's OpenGL SDL backend rendering capabilities to act as a headless "camera". The pipeline iterates through all GfxObj IDs, boots up ACViewer's rendering context, isolates the 3D model, applies a transparent background, and snapshots an isometric thumbnail. We feed these thousands of images straight into a VLM (Vision-Language Model).

Furthermore, when the VLM runs a sector generation, it triggers the engine to snap an orthographic screenshot of the map itself. The VLM acts as an **Oracle**, looking at the image and deducing visual flaws: *"The buildings are clustered too tightly on the western road, and the defensive wall has a visual breach on the north side. The foliage density is too sparse."* The VLM then issues standard API corrections through the terminal's stdin pipe. The system is essentially self-correcting its own pixel art.

Phase 5: Agentic Reasoning

(The Director)

COGNITIVE ARCHITECTURE

The system splits logic between deterministic physical constraints and stochastic LLM reasoning. The AI operates as the central **Director**, completely replacing archaic stochastic Markov chains with neural-bound logical reasoning.

- **Macro-Planning:** The user issues a natural language prompt like *"Build an elaborate Aluvian refugee camp hidden in a snowy mountain crater, it should feel desperate but fortified."* The LLM calculates the logistical steps required: it requires tents, fire pits, pathing, blocking sightlines, and defensive terrain manipulation.

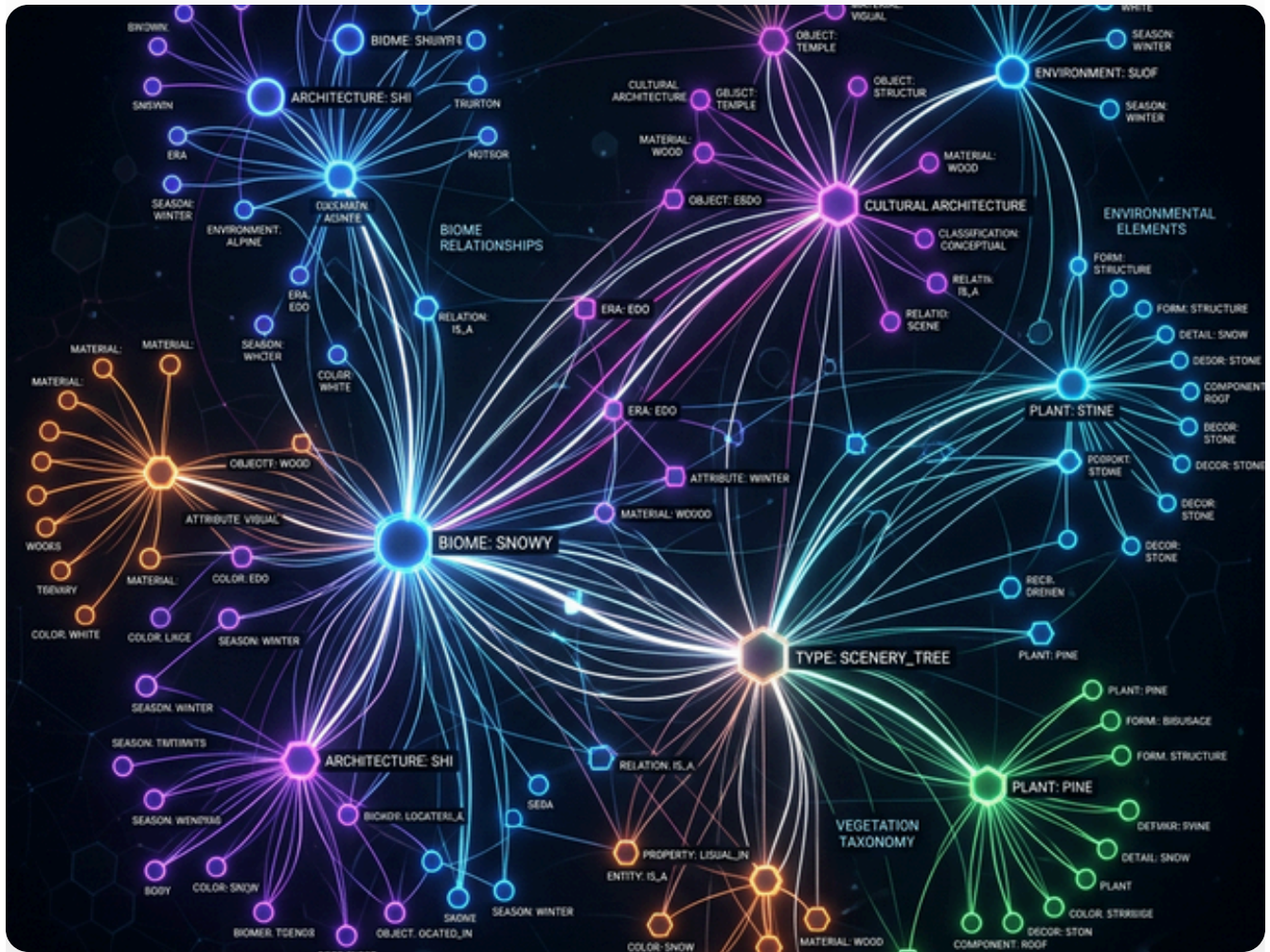
- **Constraint Translation:** The LLM looks at the available DAT metadata, checks the LSD Weenie database for appropriate "refugee" tier spawns, and issues JSON commands for specific models, deferring the precise collision bounding box alignments to the terminal's native services.
- **Iterative Correction:** If an error is hit, the prompt loop inherently provides the LLM with the terminal's JSON stack trace. The AI mathematically shifts the bounding box coordinates, evaluates the Euclidean distances, and tries again automatically.

This recursive thought process is the holy grail of our system. The AI considers the instruction, attempts a build, gets slapped down by the C# validation engine, rethinks its math, builds a successful skeleton, visualizes the result via the Screenshot Oracle, realizes it looks awful aesthetically, and refactors its own placement logic.

Phase 6: Semantic Tagging & Constraint Enforcement

AESTHETIC LAWS

If an agent can place a model perfectly, avoiding all collisions and passing all geometry verifications, it still doesn't matter if the model makes no narrative sense. Placing a tropical palm tree in a frozen snowfield passes mathematical validation, but fails aesthetic validation. The `OntologyService.cs` bridges mathematical manipulation with aesthetic cohesion.



The Auto-Classification Pipeline

We cannot manually tag tens of thousands of assets. The `ontologyService` dynamically cross-references thousands of loaded

Setup arrays against scene arrays and raw BuildingBlueprintCache data. It computes extreme bounding-box heuristics in real-time.

```
// OntologyHeuristic Computation snippet
// Deep programmatic guesses on asset categorization based on pure scale metrics
if (maxDim > 10f && partCount > 3)
    return "Structure";

// Very tall and thin – poles, trees, columns
if (aspectRatio > 3f && maxDim > 2f)
    return "Scenery";

// Small, low poly, single part – decoration / prop
if (maxDim < 1f && polyCount < 50) return
"Prop" ;
```

It then analyzes the texture files applied to those polygons. If a massive amount of the UV map involves `snow_crust.jpg`, the system computationally tags the item with `[Biome: Snowy]`.

Constraint Presets & Middleware

Overrides

With an `OntologyEntry` completely populated (e.g., tags reading `["small", "scenery", "detailed", "vertical", "snowy"]`), the terminal applies middleware blocking to the JSON engine.

If the agent context is defined as constructing a "Desert Biome", and it hallucinates an `add-object` command for a model the semantic rules classify as "Snowy Pine", the `ValidationEngine` intercepts the command. It throws a semantic error—refusing to let the agent ruin the thematic consistency of the region—and forcing the VLM to choose an appropriate regional asset.

By compiling these 6 massive phases, intertwining the heavy lifters of the AC open source community (`ACE`, `DatReaderWriter`, `ACViewer`, `Lifestoned`), the ACME `WorldBuilder` has transformed into a fully realized, programmatic engine capable of supporting the most advanced AI architectures available today. The era of manual vertex

dragging is giving way to declarative, highly validated computational genesis.